UNIVERSIDAD TECNOLÓGICA METROPOLITANA

DESARROLLO Y GESTIÓN DE SOFTWARE

OPTATIVA

QUINTAL WARNER MICHELLE ALESSANDRA

Tarea 2

INTEGRANTES:

Buenfil Yunes Julián Nahim

López Uicab Rossana Sofía

Martinez Rivero Allan Alexis

Sima Moo Jafet de Jesús

10° Cuatrimestre

2° Parcial

10-B

27 de octubre de 2026

REPO: https://github.com/Jess4lxy/StockAlert-project.git

1. Origen de la información

Se usaron datos sintéticos porque StockAlert aún no tiene negocios reales y la actividad prohíbe usar datos personales reales. El script etl/src/generate.ts simula un año de operación de una tienda (del 28 de octubre de 2025 al 27 de octubre de 2026) con 8 usuarios ficticios, 93 productos en 8 categorías y 10,000 movimientos de entrada, salida y ajuste. El stock se calcula movimiento por movimiento, así que las existencias finales y las 208 alertas de stock bajo son coherentes con el historial.

El generador usa una semilla fija: cada ejecución produce exactamente los mismos archivos. El esquema mínimo son las cuatro tablas definidas en la Tarea 1 (usuarios, productos, movimientos y alertas, en db/schema.sql). El repositorio incluye una muestra de 50 filas de cada archivo crudo en etl/data/sample; los archivos completos no se suben porque el generador los vuelve a crear idénticos.

2. Flujo ETL



1. **Extracción (generate.ts):** crea los CSV crudos. Para simular una exportación manual desde Excel, agrega 450 filas que deben descartarse y altera el formato de miles de campos.
2. **Transformación (transform.ts):** valida cada fila, normaliza los formatos, escribe CSV limpios y un reporte en etl/output/reporte_transformacion.json.
3. **Carga (load.ts):** recrea el esquema e inserta todo dentro de una sola transacción; si algo falla, no queda nada a medias.
4. **Validación (consultas.sql):** muestra el conteo final y ejecuta consultas útiles para el proyecto.

Todo el proceso se ejecuta con un solo comando: npm run etl.

3. Transformación

3.1. Problemas detectados y reglas aplicadas

Problema	Ejemplo en el CSV crudo	Regla aplicada	Filas
Duplicados	El mismo folio aparece dos veces	Se conserva la primera aparición	300 descartadas
Campos vacíos	id_producto, fecha o cantidad sin valor	Descartar (son obligatorios)	76 descartadas
Tipo de dato	cantidad = "abc" o "N/A"	Descartar	14 descartadas

Problema	Ejemplo en el CSV crudo	Regla aplicada	Filas
incorrecto			
Fecha imposible	31/02/2026 10:00	Descartar	10 descartadas
Referencia inexistente	id_producto que no está en el catálogo	Descartar	25 descartadas
Tipo de movimiento desconocido	"devolucion", "traspaso"	Descartar	25 descartadas
Fecha en formatos mezclados	21/11/2025 20:19 · 2026-01-23 11:25:00	ISO 8601 con zona -06:00 (normalizado)	2,056 corregidas
Tipo de movimiento con variantes	ENTRADA · Salida · out · Venta	entrada / salida / ajuste (normalizado)	1,456 corregidas
Cantidad como texto	"12 pzas" · "12.0" · salida "-5"	Entero positivo	699 corregidas
Observaciones en blanco	" "	Se guarda como NULL	355 corregidas
Productos	"PAPELERÍA", "papeleria" · "$12.50" · espacios de más	Categoría del catálogo, precio numérico, nombre limpio	56 corregidos; 5 duplicados

Una misma fila puede tener varios campos corregidos; por eso la suma de correcciones por campo es mayor que el número de filas corregidas.

3.2. Resultado de la transformación

Tabla	Leídos	Sin cambios	Corregidos	Descartados	Cargados
usuarios	8	7	1	0	8
productos	98	37	56	5	93
movimientos	10,450	6,090	3,910	450	10,000
alertas	208	208	0	0	208

4. Carga y validación

La carga terminó en 0.92 s y el conteo final en PostgreSQL coincide con el reporte: 8 usuarios, 93 productos, 10,000 movimientos y 208 alertas. Las restricciones del esquema (CHECK, llaves foráneas y correo único) funcionan como segunda validación: una fila inválida que escapara a la transformación haría fallar la transacción completa.

Se ejecutaron cuatro consultas de etl/sql/consultas.sql con npm run queries:

Consulta	Para qué sirve en StockAlert	Resultado	Tiempo
Productos bajo su	Saber qué reponer hoy	4 productos; p. ej., Hojas blancas	6.1 ms

Consulta	Para qué sirve en StockAlert	Resultado	Tiempo
stock mínimo		paquete 500 (5 de 10)	
Top 10 más vendidos (90 días)	Priorizar compras a proveedores	Galletas saladas 137 g, 886 unidades; Cuaderno profesional, 501	7.8 ms
Entradas y salidas por mes	Ver la tendencia del inventario	13 meses (dos parciales); los completos tienen entre 757 y 887 movimientos	17.4 ms
Alertas pendientes por categoría	Repartir el seguimiento de alertas	4 pendientes: Abarrotes, Bebidas, Higiene personal y Papelería	2.3 ms

Consulta 2. Top 10 productos más vendidos en los últimos 90 días.

```
SELECT p.nombre, p.categoria, sum(m.cantidad)::int AS unidades_vendidas, count(*)::int AS salidas
FROM movimientos m JOIN productos p ON p.id_producto = m.id_producto
WHERE m.tipo_movimiento = 'salida'
  AND m.fecha_movimiento >= (SELECT max(fecha_movimiento) FROM movimientos)
                          - interval '90 days'
GROUP BY p.id_producto, p.nombre, p.categoria
ORDER BY unidades_vendidas DESC
LIMIT 10;
```

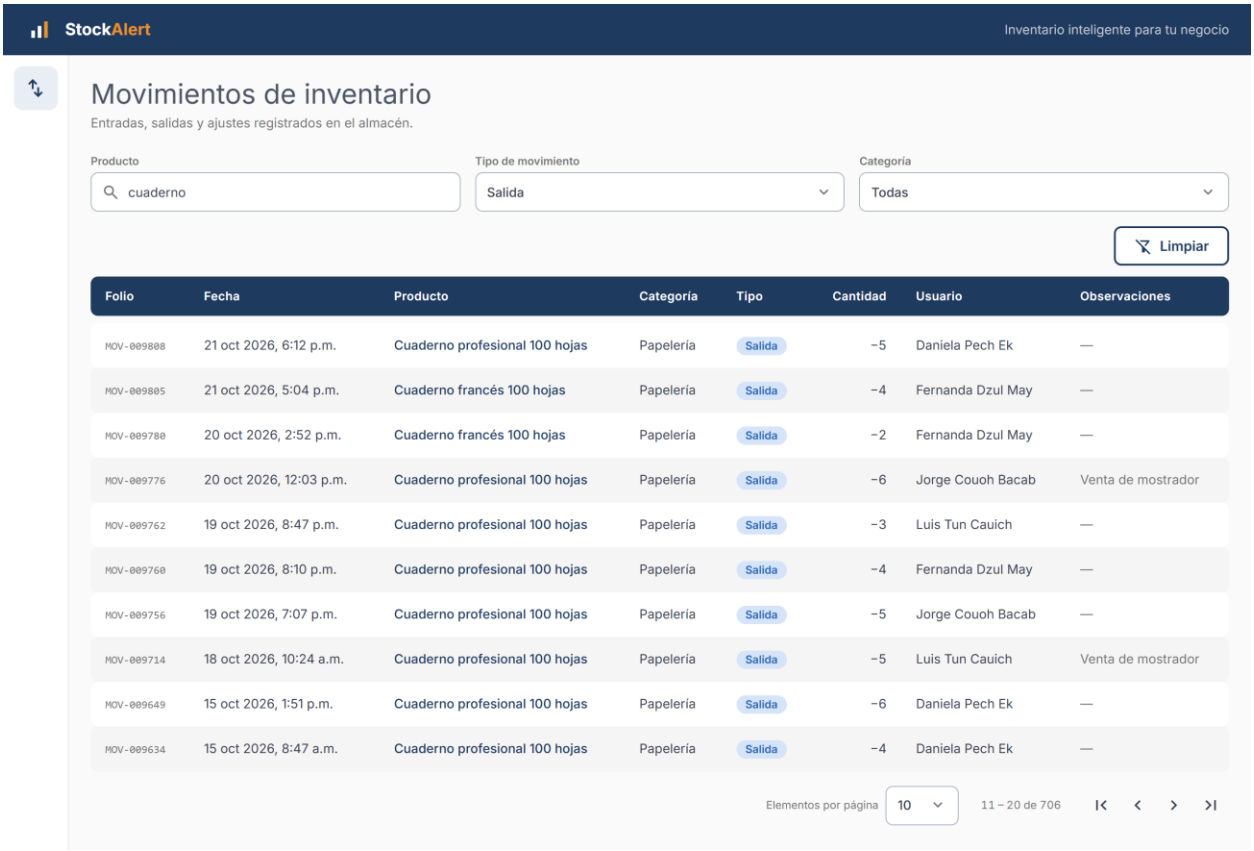
Los tiempos son de una sola ejecución en la computadora del equipo y sirven como referencia; la medición formal (cinco corridas, antes y después de optimizar) corresponde a la Tarea 3.

5. Uso desde la web

La pantalla Movimientos de inventario (Vue 3) permite buscar por nombre de producto, filtrar por tipo de movimiento y categoría, y paginar de 10 a 100 registros. La API (Node.js + Express) arma la consulta con parámetros ($1, $2…) y resuelve la búsqueda, el filtro y la paginación en PostgreSQL con LIMIT y OFFSET, así que el navegador solo recibe la página que muestra.

Ejemplo: la petición GET /api/movimientos?q=cuaderno&tipo=salida&page=2&limit=10 devuelve 10 filas y el total de coincidencias (706) en unos 2.5 KB; enviar los 10,000 movimientos completos ocuparía unos 2.4 MB, casi mil veces más.

Búsqueda "cuaderno" con filtro Salida, página 2 de 71 (10 por página).



Vista en celular: cada fila se muestra como tarjeta.



6. Reproducción y seguridad

1. Copiar .env.example a .env en la raíz del repositorio y cambiar la contraseña.
2. Levantar PostgreSQL con docker compose up -d.
3. Dentro de etl/, ejecutar npm install y npm run etl.
4. Levantar la API y la web con npm run dev en backend/ y en frontend/.

Los pasos completos están en el README. No se publican credenciales: .env está en .gitignore y el repositorio solo contiene valores de ejemplo. Se usa PostgreSQL local en Docker porque el plan gratuito de Render elimina la base de datos a los 30 días.

Conclusión

El proceso ETL es reproducible de principio a fin: el mismo comando genera, limpia y carga los mismos 10,000 movimientos y deja un reporte de lo que se corrigió (3,910 filas) y de lo que se descartó (450, cada una con su motivo). La aplicación web trabaja sobre ese volumen sin descargarlo completo, gracias a la paginación del lado del servidor.

Aun así, la búsqueda por nombre y los filtros recorren la tabla completa porque todavía no existen índices secundarios. Esa será la línea base para medir la optimización en la Tarea 3.